

Accesso ai dati da software

Gianluca Cima, Maurizio Lenzerini



SAPIENZA
UNIVERSITÀ DI ROMA

Anno accademico 2024/2025

In genere:

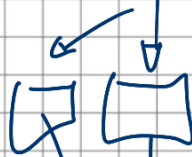
LIVELLO
DATA
PRESENTAZIONE

PROLOGO



Quella che noi facciamo con le query

LIVELLO
DELL'APPLICAZIONE



TUTTI I MAPPEL SOFTWARE
CIA SONO IN LA LINGUA DEL
SOFTWARE → CASA DIVE AND
PROGRAMM IL SOFTWARE

LIVELLO
DEI DATI



Tabelle,
viste

ALTRE COSE

QUALI DATI DI UNA
USARE IL SOFTWARE

SALTA IL LIVELLO DI
APPLICAZIONE.
NON TUTTI GLI UTILI
CIA MA IL PERMANO DI
PARLA

IL 90% DEI CASI NON C'E' IL CASO DI UTILITA' → DBMS

Qua infatti troviamo di accesso ai dati da software, quindi passare dal livello applicazione.

• Vedremo Java ma andrebbe bene anche qualsiasi linguaggio.

• Vedremo anche che ci sono anche "Servizi" dentro i DBMS e questi si chiamano STORED PROCEDURE

Pezzo di SOFTWARE DENTRO IL DBMS e che può essere invocata sin dentro una query che anche in altri casi

Perché è possibile? perché è utile mettere dei servizi che possono essere usati in tutte le applicazioni

Esempi:

"Come si calcola il prezzo di un prodotto in una certa data per una certa classe di clienti"

Richiede un algoritmo che in genere viene usato nel DBMS e non a livello applicativo. "SOFTWARE CLOSE TO DATA"

Quando noi scriviamo un servizio a livello applicativo, possiamo OTTIMIZZARE l'accesso ai dati? Dobbiamo pensare noi a come ottimizzare l'accesso ai dati ma il DBMS ha tutti gli elementi per ottimizzare. Allora abbiamo 2 ragioni per cui ha senso:

- 1) Progettare Servizi riutilizzabili da chiunque e CONCRETE
- 2) Cogliere che il DBMS fa il lavoro di OTTIMIZZAZIONE che a livello applicativo non si fa o non conviene

Avere accesso ai dati di Software in 2 modi:

- INTERNO: attraverso lo STANDARD PROCEDURE che mi realizza dentro il DBMS
- ESTERNO: a livello di applicazione. In questa caso ci affidiamo ad un protocollo JDBS, cioè delle librerie Java da installare di accesso al DBMS

Altre informazioni:

OGNI DBMS ha il suo linguaggio. I SIMILI. POSTGRES ha psSQL

STESSA
PRATICA
COMPATIBILITA'
DI TURIN

→ TURIN COMPATIBLE
SQL in genere
ma lo è

Overview

- 1 Introduzione
- 2 Accesso da software interno
- 3 Accesso da software esterno

Overview

- 1 Introduzione
- 2 Accesso da software interno
- 3 Accesso da software esterno

Accesso da software

Finora abbiamo analizzato il caso in cui alla base di dati accede un utente umano utilizzando SQL. Nella pratica, però, nasce spesso l'esigenza di accedere ai dati da parte di moduli software, e quindi con meccanismi che sono propri dei linguaggi di programmazione. I DBMS moderni offrono due possibilità per fare questo:

- 1 Associare alla base di dati dei “moduli software” definiti all'interno del DBMS stesso e quindi nel data layer. In questo caso si parla di **accesso da software interno** ed i moduli possono essere scritti in un linguaggio di programmazione qualunque oppure in un linguaggio proprietario del DBMS (nel caso di PostgreSQL: PL/pgSQL). Questi moduli software sono poi invocati da utenti umani o da altri moduli software, sia interni sia esterni.
- 2 Accedere alla base di dati da software esterno (**accesso da software esterno**). Nell'architettura del software a tre livelli, questo corrisponde ad accedere al data layer dall'application layer. Siccome l'application layer è costituito da moduli software, nasce l'esigenza di accedere ai dati con meccanismi che sono propri dei linguaggi di programmazione. Tutti i linguaggi di programmazione, come ad esempio Python, Java, C, C++, consentono questo accesso. Qui noi studiamo il meccanismo offerto da Java per accedere a basi di dati relazionali, ossia quello che si chiama il protocollo JDBC.

Overview

- 1 Introduzione
- 2 **Accesso da software interno**
- 3 Accesso da software esterno

PL/pgSQL

- Noi illustriamo le caratteristiche salienti di PL/pgSQL, che è un linguaggio di programmazione procedurale supportato da PostgreSQL. Ma esistono altri linguaggi analoghi nei vari DBMS.
- PL/pgSQL, includendo la possibilità di usare cicli e strutture di controllo avanzate, è stato creato in modo da svolgere operazioni complesse che non sono propriamente in linea con la filosofia e le potenzialità di SQL.
- I moduli software creati in questo linguaggio sono chiamati “function” (noti anche come **stored procedure**). Una volta che si fa compilare una function al DBMS, essa viene memorizzata dal sistema nel contesto del database in cui viene definita, e può essere chiamata in qualsiasi momento come parte di un’istruzione SQL, o attivata da un **trigger** (vedere dopo).
- Il compilatore di PL/pgSQL è installato di default su PostgreSQL.

PL/pgSQL: sintassi di base per una stored procedure

L'istruzione per creare una "function" o "stored procedure" è la **CREATE FUNCTION** (oppure **CREATE OR REPLACE FUNCTION**)

```
CREATE FUNCTION function_name (arg1 arg_type, ..., argn arg_type)
RETURNS {return_type | TABLE (col1 col_type, ...,) | TRIGGER} AS
$$
```

DECLARE

<dichiarazioni di variabili>

BEGIN

<istruzioni>

END:

```
$$ LANGUAGE plpgsql;
```

Se *return_type* è VOID, allora la function è in realtà una procedura.

Per maggiori dettagli si veda la documentazione ufficiale: <https://www.postgresql.org/docs/9.6/static/plpgsql-structure.html>.

Stored Procedure: esempio - parte 1

Assumiamo di avere la seguente relazione:

fattura(soggetto: VARCHAR(100), imponibile: REAL, aliquota: INTEGER)
 $\{\langle a, 1000.00, 22 \rangle, \langle a, 30.00, 4 \rangle, \langle b, 10.000, 10 \rangle\}$

ottenuta tramite:

```
CREATE TABLE fattura (soggetto VARCHAR(100), imponibile REAL, aliquota INTEGER);
```

```
INSERT INTO Fattura VALUES ('a', 1000.00, 22), ('a', 30.00, 4), ('b', 10.000, 10);
```

Supponiamo adesso di voler calcolare, dato un soggetto, una tabella con una riga che indichi tale soggetto ed il totale che esso dovrà versare di tasse. Lo possiamo fare attraverso la seguente stored procedure:

← cada frase - possa ser ativada um S2R, PT

R RASTRUSCO VNA QVLEY

TUTTA QUESTA PENSIERA
NON LA POSSO FARE COME LE
ALTRE NORMALMENTE

- Esempio di chiamata: `SELECT * from totale_fattura('a');` *query normale*
- Se il tipo di ritorno di `f( ... )` è `VOID`, la chiamata è `SELECT f( ... );`

Stored Procedure: beneficial

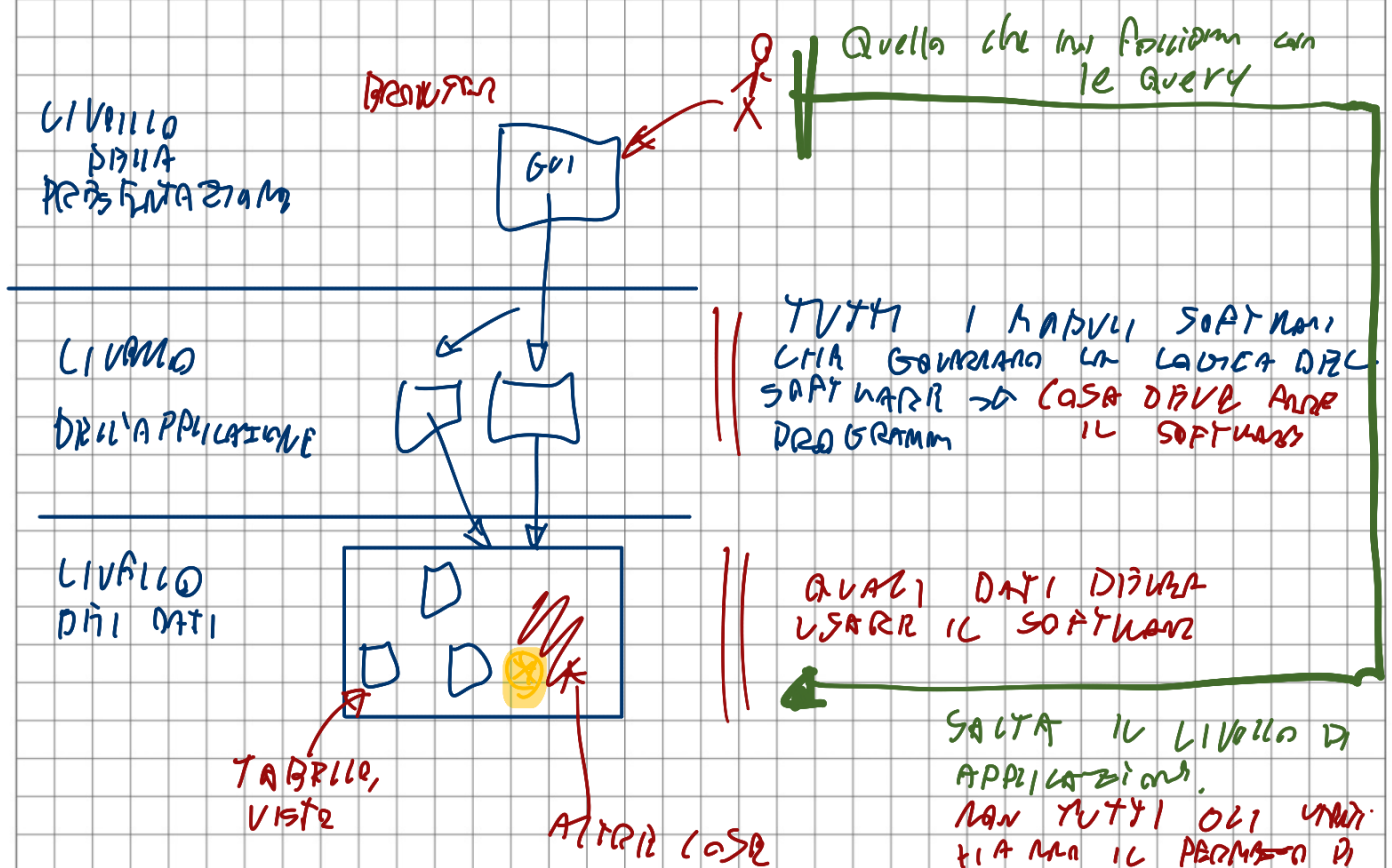
Molte delle cose che si fanno con le stored procedure possono essere fatte direttamente con le query.

Tuttavia, il vantaggio di definire una stored procedure, oltre al loro uso con i trigger (si veda dopo), è proprio quello di evitare che gli utilizzatori della base di dati (utenti umano o moduli software) scrivano direttamente le query.

Forzare gli utenti ad usare le stored procedure al posto delle query ha i seguenti vantaggi:

- la base di dati si può meglio proteggere da eventuali usi maligni o errati da parte degli utilizzatori;
- con le stored procedure si possono imporre comportamenti coerenti con la semantica dei dati, evitando che siano aggirati da chi usa la base di dati.

RILEGGERE



SU PPAIARNA che in realtà conta quante volte qualcuno prende alla mia cupola della FORTUNA, dove:

FATTURA

oggetto	imparabile	aliquota

Cioè:

CONTROLLI

soys.	Totale
x	100
y	200

Voglio quindi registrare il soggetto in una tabella di parte, quante volte è stato controllato una certa tupla della mia tabella FATTURA.

Significa che quando siamo a livello applicativo, ogni volta dovei modificare la tabella Controlli.

Questa cosa non si può fare. E' possibile solo in situazioni molto particolari.

Allora come era questa situazione? Cioè che l'aggiornamento della tabella Controlli deve succedere a livello dei Dati, poiché è molto più vicino al DBMS perché così. IL DBMS deve quindi sapere rilevare l'accesso alla tupla FATTURA e modificare la tupla Controlli. Va bene anche allora il Totale di ogni valore che si aggiunge una tupla della FATTURA. Quindi ogni valore che succede potrebbe essere invece in servizio.

Concetto di TRIGGER: quando succede un certo evento su una tabella, chiama una STORED PROCEDURE/Procedura.

PROVA + SEIROS VIM PRODUZIR O PDA

- Un **trigger** è un meccanismo per specificare che una certa stored procedure deve essere eseguita in maniera automatica in coincidenza di un determinato evento, in particolare l'inserimento, la cancellazione o l'aggiornamento di una tupla da una relazione.
- Viene associato ad una tabella e viene attivato in modo automatico quando il determinato evento relativo a tale tabella avviene

```
CREATE TRIGGER trigger_name [BEFORE | AFTER | INSTEAD OF]
[INSERT | DELETE | UPDATE] ON table_name
[FOR EACH ROW | FOR EACH STATEMENT] EXECUTE
PROCEDURE procedure_name;
```

dove la procedura *procedure_name* deve avere come ritorno il tipo TRIGGER.

Trigger: semantica

- BEFORE | AFTER | INSTEAD OF: Un trigger può essere attivato prima (BEFORE) che l'operazione di modifica avvenga, oppure dopo (AFTER) che l'operazione sia stata eseguita, o può essere attivato in sostituzione dell'operazione stessa (INSTEAD OF, solo sulle viste).
- INSERT | DELETE | UPDATE: L'evento che attiva il trigger.
- FOR EACH ROW | FOR EACH STATEMENT:
 - Se il trigger viene definito FOR EACH ROW, allora viene attivato per ognuna delle tuple coinvolte nell'operazione di modifica;
 - se invece il trigger viene definito FOR EACH STATEMENT (default), allora viene attivato una sola volta per ciascuna operazione di modifica, indipendentemente dalle tuple coinvolte.



In SQL, i trigger `INSTEAD OF` vengono utilizzati principalmente in contesti in cui si desidera intercettare e modificare il comportamento di un'operazione DML (Data Manipulation Language) come `INSERT`, `UPDATE` o `DELETE` su una vista (view) invece che su una tabella. Questo tipo di trigger è particolarmente utile quando si lavora con viste che non sono aggiornabili direttamente, come le viste che coinvolgono join o aggregazioni.

Apprendimento
INSTEAD OF

Ecco alcune occasioni in cui si potrebbe utilizzare un trigger `INSTEAD OF`:

1. **Viste Non Aggiornabili:** Quando si ha una vista che non può essere aggiornata direttamente a causa della sua complessità (ad esempio, viste che coinvolgono join tra più tabelle o viste con aggregazioni), un trigger `INSTEAD OF` può essere utilizzato per definire come gestire le operazioni di `INSERT`, `UPDATE` o `DELETE` su quella vista.
2. **Logica Complessa:** Quando si desidera eseguire una logica complessa o personalizzata durante un'operazione DML su una vista. Ad esempio, potrebbe essere necessario aggiornare più tabelle o eseguire calcoli complessi che non possono essere gestiti direttamente dalla vista.
3. **Controllo degli Accessi:** Per implementare controlli di sicurezza o validazioni aggiuntive che non possono essere gestiti direttamente dalla vista. Ad esempio, si potrebbe voler verificare che un utente abbia i permessi necessari per eseguire un'operazione di aggiornamento su una vista.

Ecco un esempio di come definire un trigger `INSTEAD OF` su una vista:

```
-- Creazione delle tabelle di esempio
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50),
    DepartmentID INT
);

CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName NVARCHAR(50)
);

-- Creazione della vista
CREATE VIEW EmployeeDepartmentView AS
SELECT e.EmployeeID, e.FirstName, e.LastName, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID;

-- Creazione del trigger INSTEAD OF INSERT
CREATE TRIGGER trgInsteadOfInsert ON EmployeeDepartmentView
INSTEAD OF INSERT
AS
BEGIN
    -- Logica personalizzata per l'inserimento
    INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID)
    SELECT EmployeeID, FirstName, LastName, d.DepartmentID
    FROM inserted i
    JOIN Departments d ON i.DepartmentName = d.DepartmentName;
END;

-- Creazione del trigger INSTEAD OF UPDATE
CREATE TRIGGER trgInsteadOfUpdate ON EmployeeDepartmentView
INSTEAD OF UPDATE
AS
BEGIN
```

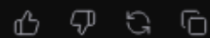
```
-- Logica personalizzata per l'aggiornamento
UPDATE Employees
SET FirstName = i.FirstName,
    LastName = i.LastName,
    DepartmentID = d.DepartmentID
FROM inserted i
JOIN Departments d ON i.DepartmentName = d.DepartmentName
WHERE Employees.EmployeeID = i.EmployeeID;

END;

-- Creazione del trigger INSTEAD OF DELETE
CREATE TRIGGER trgInsteadOfDelete ON EmployeeDepartmentView
INSTEAD OF DELETE
AS
BEGIN
    -- Logica personalizzata per la cancellazione
    DELETE FROM Employees
    WHERE EmployeeID IN (SELECT EmployeeID FROM deleted);
END;
```

In questo esempio, i trigger `INSTEAD OF` vengono utilizzati per gestire le operazioni di `INSERT`, `UPDATE` e `DELETE` sulla vista `EmployeeDepartmentView`, che non è direttamente aggiornabile a causa del join tra le tabelle `Employees` e `Departments`.

17:07



VISTE vs TABELLE



Utilizzare una vista al posto di una tabella può avere senso in diverse situazioni, a seconda delle esigenze specifiche del progetto e delle caratteristiche del database. Ecco alcune circostanze in cui l'uso di una vista può essere vantaggioso:

1. Semplificazione delle Query:

- **Riduzione della Complessità:** Le viste possono semplificare le query complesse. Ad esempio, se una query coinvolge join tra più tabelle o aggregazioni, creare una vista può rendere più semplice e leggibile l'accesso ai dati.
- **Riutilizzo del Codice:** Le viste permettono di riutilizzare query complesse in diverse parti dell'applicazione senza dover riscrivere il codice SQL.

2. Sicurezza e Controllo degli Accessi:

- **Restrizione degli Accessi:** Le viste possono essere utilizzate per limitare l'accesso ai dati sensibili. Ad esempio, si può creare una vista che mostra solo un sottoinsieme delle colonne di una tabella, nascondendo le informazioni sensibili.
- **Controllo degli Accessi:** Le viste possono essere utilizzate per implementare politiche di sicurezza più granulari, permettendo a diversi utenti di accedere a diverse porzioni dei dati.

3. Astrarre la Complessità del Database:

- **Nascondere la Struttura del Database:** Le viste possono essere utilizzate per nascondere la struttura interna del database agli utenti finali o alle applicazioni, fornendo un'interfaccia più semplice e stabile.
- **Compatibilità:** Le viste possono aiutare a mantenere la compatibilità con le applicazioni esistenti quando la struttura del database cambia. Ad esempio, se si rinomina una colonna o si suddivide una tabella, una vista può mantenere la stessa interfaccia per le applicazioni esistenti.

4. Ottimizzazione delle Prestazioni:

- **Indicizzazione:** Alcuni database permettono di creare indici sulle viste, migliorando le prestazioni delle query.
- **Materializzazione:** Le viste materializzate possono essere utilizzate per memorizzare i risultati di query complesse, migliorando le prestazioni delle query ripetitive.

5. Reporting e Analisi:

- **Reporting:** Le viste possono essere utilizzate per creare report e dashboard, fornendo una rappresentazione dei dati più adatta alle esigenze di reporting.
- **Analisi:** Le viste possono essere utilizzate per creare aggregazioni e calcoli complessi che facilitano l'analisi dei dati.

6. Gestione dei Dati Storici:

- **Versioni dei Dati:** Le viste possono essere utilizzate per gestire diverse versioni dei dati, ad esempio per mantenere una vista storica dei dati mentre si lavora su una versione attuale.

Trigger: osservazioni importanti

- Se il trigger è attivato da una istruzione che è a sua volta una transazione (INSERT, DELETE, UPDATE, ad esempio) o che appare in una transazione, le operazioni nell'ambito del trigger sono considerate appartenenti a tale transazione.
- Se il trigger è definito **BEFORE e FOR EACH ROW**, allora
 - se il valore v che la procedura invocata dal trigger che opera sulla tupla t restituisce è **NULL**, allora l'operazione che ha scatenato il trigger **non viene eseguita** sulla tupla t ,
 - se invece non è **NULL**, allora è di tipo record (molte volte sarà banalmente **NEW**) e in questo caso l'operazione che ha scatenato il trigger **viene eseguita** ed usa come tupla t proprio v .
- Se il trigger è definito **AFTER o FOR EACH STATEMENT**, il valore di ritorno è ignorato.

Regole predefinite:

Quando inserisci una nuova Email di posta Falla allora
se 3 sogg allora UP DATE, se 4 sogg inserisci una Email
dentro Controllo. Se cancelli dalla UP DATE, se aggiungi l'indirizzo
dopo fare UP DATE

Trigger: esempio - parte 1

Assumiamo di avere i seguenti schemi:

```
lavoratore(codice: numeric, nome: varchar(20), cognome: varchar(20),  
           inprogetto: numeric, salario: int);  
progetto (codice: numeric, usura: numeric, responsabile: numeric);
```

ed assumiamo che al momento dell'inserimento di un nuovo progetto, questo lo si debba assegnare a tutti i lavoratori che hanno il campo "inprogetto" pari a NULL, cioè i lavoratori che sono senza un progetto oppure sono tali che non si conosca il progetto a cui lavorano.

TRIGGIR

Trigger: esempio - parte 3

Decidiamo che quando si aumenta l'usura di un progetto si aumenti il salario del 10% a tutti i lavoratori coinvolti in quel progetto.

```
CREATE OR REPLACE FUNCTION usura_progetto() RETURNS TRIGGER
AS
$$
```

BEGIN

IF (NEW.usura > OLD.usura) THEN

UPDATE lavoratore

```
SET salario = salario * 1.1 WHERE inprogetto = NEW.codice;
```

END IF;

```
RETURN NEW;
```

END;

```
$$ language plpgsql;
```

```
CREATE TRIGGER trigger_piu_usura_progetto AFTER UPDATE ON progetto
FOR EACH ROW EXECUTE PROCEDURE usura_progetto();
```

Trigger, altro esempio: chiusura transitiva di un grafo

Supponiamo di avere la tabella grafo che rappresenta gli archi di un grafo G (f sta per “from” e t sta per “to”), definita tramite:

```
CREATE TABLE grafo (
    f varchar,      t varchar,      PRIMARY KEY (f,t)
);
```

e la tabella `reachable` definita tramite:

```
CREATE TABLE reachable (
  f varchar,      t varchar,      PRIMARY KEY (f,t)
);
```

Vogliamo calcolare, ad ogni inserimento di un arco in G (ossia di una tupla nella tabella grafo), la **chiusura transitiva** del grafo G e memorizzarla nella tabella `reachable` (ovvero, vogliamo sempre avere in `reachable` tutte e sole le tuple $\langle x, y \rangle$ tali che y è raggiungibile da x mediante un cammino nel grafo).

Chiusura transitiva di un grafo - Parte 1

```
CREATE OR REPLACE FUNCTION compute_transitive_closure() RETURNS TRIGGER AS
$$ DECLARE edgeobject record; leastfixpoint bool; r record;
BEGIN
  IF (NEW.f,NEW.t) not in (SELECT f,t FROM reachable)
  THEN INSERT INTO reachable VALUES (NEW.f, NEW.t);
    leastfixpoint := false;
    WHILE NOT leastfixpoint LOOP
      leastfixpoint := true;
      FOR r IN SELECT * FROM reachable LOOP
        FOR edgeobject IN SELECT * FROM reachable LOOP
          IF (r.t = edgeobject.f)
          THEN IF NOT EXISTS (SELECT * FROM reachable WHERE
                                reachable.f=r.f AND reachable.t=edgeobject.t)
          THEN leastfixpoint := false;
              INSERT INTO reachable VALUES (r.f, edgeobject.t);
            END IF;
          END IF;
        END LOOP;
      END LOOP;
    END LOOP;
  END IF;
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```


ESEMPIO IN CLASSE lezione 29-30

La base di dati contiene la tabella "corso" (con il tipo, che è "informatica" o "altro"), la tabella "studente" e la tabella "esame", che riporta gli esami superati dagli studenti.

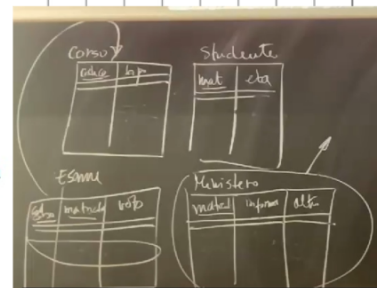
In più, la tabella "ministero" ha lo scopo di comunicare i dati al Ministero. Essa, quindi, deve essere costantemente aggiornata: per ogni studente deve riportare il numero di esami di informatica e di altre materie superati dallo studente.

Sappiamo che non ci saranno cancellazioni né aggiornamenti della tabella "esame", né inserimenti o aggiornamenti diretti di "ministero".

```
create table corso (  
  codice integer,  
  tipo varchar constraint sutipo  
    check (tipo in ('informatica', 'altro')) not null,  
  constraint corsokey primary key (codice)  
);  
  
create table studente (  
  matricola varchar constraint studentekey primary key,  
  eta integer  
);  
  
create table esame (  
  codcorso integer constraint fkesame1 references corso  
    on delete restrict on update restrict,  
  matricola varchar constraint fkesame2 references studente  
    on delete restrict on update restrict,  
  voto integer,  
  constraint esamekey primary key (codcorso, matricola)  
);  
  
create table ministero (  
  matricola varchar constraint ministerokey primary key,  
  informatica integer default 0,  
  altro integer default 0  
);
```

Allo scopo di tenere la tabella "ministero" aggiornata, dobbiamo gestire sia l'inserimento di un nuovo studente (che deve causare l'inserimento di una nuova tupla nella tabella "ministero"), sia l'inserimento di un nuovo esame (che deve causare l'aggiornamento della tupla nella tabella "ministero" relativa allo studente che ha sostenuto l'esame). Definiamo quindi due opportuni trigger che vengono scatenati da questi due tipi di inserimento.

```
create or replace function aggiungiAministero()  
returns trigger as  
$$  
begin  
  insert into ministero(matricola) values (new.matricola);  
  return new;  
end  
$$ LANGUAGE plpgsql;  
  
create or replace trigger triggerInsertStudente after insert on studente  
for each row execute procedure aggiungiAministero();  
  
create or replace function aggiornaMinistero()  
returns trigger as  
$$  
begin  
  if (new.codcorso in  
    (select codice from corso where tipo = 'informatica'))  
  then update ministero set informatica = informatica + 1  
    where matricola = new.matricola;  
  else update ministero set altro = altro + 1  
    where matricola = new.matricola;  
  end if;  
  return new;  
end  
$$ LANGUAGE plpgsql;  
create or replace trigger triggerInsertEsame after insert on esame  
for each row execute procedure aggiornaMinistero();
```



Supponiamo che non vogliamo che il ministro perda delle tuple, possiamo:

create or replace function dontTouchMinistro()

returns trigger as

\$\$

begin

return NULL; || quando, chi fa la delete, esegue la delete, ottiene tutto l'operazione ma l'operazione di ritorno arriva su NULL cioè non su NEW.

end

\$\$ LANGUAGE plpgsql;

create or replace trigger triggerDontTouchMinistro before delete on Ministro
for each row execute procedure dontTouchMinistro();

Con un trigger posso eliminare ogni idea di DELETE o UPDATE su una tabella

Chiusura transitiva di un grafo - Esercizi

Si noti che nella soluzione presentata abbiamo optato per un trigger di tipo "FOR EACH ROW" e questo significa che per ogni tupla inserita nel grafo verrà invocata la stored procedure. L'aspetto non proprio soddisfacente è che essa calcola ex-novo **tutta** la chiusura transitiva e non solo la porzione che dipende dagli archi inseriti.

- **Esercizio 1.** Se vogliamo insistere con l'approccio di calcolare ex-novo la chiusura transitiva, allora possiamo cambiare la definizione del trigger in modo che sia di tipo "FOR EACH STATEMENT", cosicchè sia invocata una volta per statement (invece che per row). Cambiare opportunamente sia la stored procedure sia il trigger al fine di seguire questa strada.
- **Esercizio 2.** Se vogliamo cambiare approccio, possiamo confermare la decisione di definire il trigger "FOR EACH ROW", ma cambiare la stored procedure in modo che essa calcoli solo la porzione della chiusura transitiva che è rilevante rispetto alle tuple inserite. Cambiare opportunamente la stored procedure al fine di seguire questa strada.

Overview

- 1 Introduzione
- 2 Accesso da software interno
- 3 Accesso da software esterno

II framework JDBC

- L'accesso ad una base di dati relazionale dall' "application layer" avviene mediante l'uso di interfacce (API) standard per la comunicazione con DBMS, come ODBC o **JDBC (Java Data Base Connectivity)**
- JDBC è un'API Java per l'esecuzione di comandi SQL indirizzati ad un DBMS: serie di classi ed interfacce che implementano una modalità standardizzata per l'interazione con il DBMS da parte di applicazioni Java
- Ciascun DBMS fornisce un driver specifico che:
 - deve essere caricato a run-time dal programma che vuole usare il DBMS
 - traduce le chiamate alle funzioni JDBC in chiamate alle funzioni del DBMS

Caratteristiche di JDBC

- esecuzione di comandi SQL
 - ① DDL (Data Definition Language)
 - ② DML (Data Manipulation Language)
- manipolazione dei risultati tramite **result set** (una forma di cursore)
- reperimento di **metadati**
- gestione di **transazioni**
- gestione di **errori** ed **eccezioni**
- definizione di **stored procedure** scritte in Java (supportate da alcuni DBMS, previa installazione di specifici package - noi non affronteremo questo aspetto, perché per l'accesso interno usiamo PL/pgSQL)

Driver JDBC

Un driver JDBC per un certo DBMS viene distribuito in genere dalla casa produttrice del DBMS. È di fatto una libreria Java che va opportunamente linkata in fase di esecuzione dell'applicativo. Questo va fatto:

- (i) settando opportunamente le variabili d'ambiente
- (ii) indicando da riga di comando la libreria da linkare quando si lancia l'applicazione
- (iii) configurando l'ambiente di sviluppo Java che si sta usando.

Elementi di JDBC

Come abbiamo già detto, JDBC è un API Java di connessione a dati residenti in DB relazionali e consiste di un insieme di elementi (classi e interfacce) scritte nel linguaggio di programmazione Java (si trovano nel package `java.sql` appunto).

I più importanti elementi di JDBC sono:

- la classe Driver
- la classe DriverManager
- l'interfaccia Connection
- la classe DatabaseMetaData
- l'interfaccia Statement
- l'interfaccia ResultSet
- l'interfaccia ResultSetMetaData
- l'interfaccia PreparedStatement
- l'interfaccia CallableStatement

Fasi dell'interazione con una base di dati via JDBC

L'interazione con una base di dati via JDBC prevede le seguenti fasi di esecuzione nell'ambito del programma software:

- 1 Caricamento dinamico della classe corrispondente al driver JDBC opportuno per la sorgente dati e registrazione del driver presso il driver manager
- 2 Connessione con la sorgente di dati, stabilita attraverso il driver manager
- 3 Sessione di interazione attraverso la connessione, in cui si definiscono gli "statement" e si eseguono tali statement comunicandoli alla sorgente
- 4 Chiusura della connessione

Prima fase: caricamento del driver JDBC

- Il metodo statico `forName` della classe `Class` carica dinamicamente la classe Java specificata nella stringa passata come parametro, che indica il nome completamente qualificato (cioè con il package a cui appartiene) della classe stessa.
- Nel nostro caso, passiamo una stringa per la creazione di un oggetto della classe `Driver` specifico per il DBMS selezionato, il quale automaticamente “registra” se stesso con la classe `DriverManager`.

Ad esempio:

```
Class.forName("oracle.jdbc.pool.PooledConnection");
```

```
Class.forName("com.mysql.jdbc.Driver");
```

```
Class.forName("org.postgresql.Driver");
```

Seconda fase: connessione con la sorgente dati

- Per parlare ad una sorgente, ovvero un DBMS, occorre farlo attraverso una connessione. La connessione si stabilisce attraverso il metodo **statico** `getConnection` della classe `DriverManager`. Quest'ultimo restituisce un oggetto di una classe che implementa l'**interfaccia** `Connection` in maniera **specificata** per il DBMS per il quale si è precedentemente caricato il driver a run-time
- I parametri di `getConnection` sono un'**URL JDBC**, lo **username** e la **password**, dove l'URL JDBC ha la forma `jdbc:<sub-protocollo>:<altri-parametri>`

Seconda fase: connessione con la sorgente dati

```
Class.forName("org.postgresql.Driver");
```

```
String url="jdbc:postgresql:test";
```

```
String username="root";
```

```
String password="";
```

```
Connection conn=DriverManager.getConnection(url,  
                                             username,password);
```

- postgresql è il subprotocollo per PostgreSQL; test è il nome del database a cui ci si vuole connettere (può anche mancare).

Seconda fase: la classe DatabaseMetaData

Permette di ottenere informazioni sul sistema di basi di dati relativo ad una connessione, come ad esempio informazioni sul catalogo. Il codice seguente mostra come ottenere nome e versione di un driver JDBC:

```
DatabaseMetaData md=conn.getMetaData();  
System.out.println(" Informazioni sul driver:");  
System.out.println(" Nome: " + md.getDriverName() +  
                    "; versione: " + md.getDriverVersion());
```

Si noti che il metodo `getMetaData()` è invocato tramite un oggetto di tipo `Connection`.

Seconda fase: alcuni metodi della classe DatabaseMetaData

- `getConnection()` restituisce l'oggetto di tipo `Connection` a cui si riferisce l'oggetto di classe `DatabaseMetaData`
- `getURL()` restituisce l'URL per il database in uso
- `getUserName()` restituisce il nome dell'utente connesso
- `getMaxConnections()` restituisce il numero massimo di connessioni possibili

Terza fase: sessione di interazione

L'interazione con la sorgente di dati avviene attraverso l'uso di oggetti di classi che, per lo specifico DBMS che si accede, implementano le seguenti interfacce:

- **Statement**: istruzione SQL nota a tempo di compilazione
- **PreparedStatement**: istruzione SQL la cui struttura è fissata a tempo di compilazione, ma che può ammettere parametri il cui valore può variare a run-time
- **CallableStatement**: istruzione necessaria per accedere alle stored procedures
- **ResultSet**: struttura dati simile ad un cursore

Terza fase: Statement

- Gli oggetti che consentono di eseguire una istruzione SQL sono istanze di classi **statement** che implementano l'interfaccia `Statement`
- Uno statement è sempre **relativo ad una connessione** $\Rightarrow$ serve una istanza di una connessione attiva per predisporre uno statement (metodo `createStatement` di `Connection`)

Ad esempio:

```
Statement stmt=conn.createStatement();
```

Terza fase: metodi importanti della classe Statement

- `executeUpdate` per eseguire comandi di creazione, eliminazione, aggiornamento
- `executeQuery` per eseguire interrogazioni
- Entrambi prendono in ingresso una stringa, che di fatto è la stringa corrispondente al comando SQL che si vuole passare al DBMS (senza il punto e virgola finale!) e che il DBMS eseguirà.

Attenzione!

- Java è case-sensitive, mentre SQL lo è solamente per i valori di tipo stringa
- In Java gli apici che delimitano stringhe sono doppi, mentre in SQL sono singoli

Journal of Management Education 36(7) 809–824

im Prozess stand

MI Preparo la staza

0 1 0

Terza fase: executeUpdate

Creiamo la tabella COFFEES con la stringa SQL di prima:

```
stmt.executeUpdate(createTableCoffees);
```

Inseriamo due tuple in COFFEES; si osservi che in questo caso **utilizziamo sempre lo stesso oggetto Statement per le diverse operazioni.**

```
stmt.executeUpdate("INSERT INTO COFFEES " + "VALUES  
( 'Colombian',101,7.99,0,0)" );
```

```
stmt.executeUpdate("INSERT INTO COFFEES " + "VALUES  
( 'Espresso',150,9.99,0,0)" );
```

Il metodo `executeUpdate` ha l'effetto di eseguire il comando rappresentato dalla stringa passata come parametro e restituisce un `int` pari al numero di tuple modificate. Nel caso di uno statement DDL, ad esempio la creazione di una tabella, viene restituito 0.

Terza fase: executeQuery

Il metodo `executeQuery` di `Statement` ha l'effetto di eseguire il comando `select` rappresentato dalla stringa passata come parametro e restituisce un oggetto di tipo `ResultSet`

```
ResultSet rs=stmt.executeQuery("SELECT * FROM COFFEES");
```

o anche

```
String query="SELECT COF_NAME,PRICE FROM COFFEES";
```

```
ResultSet rs=stmt.executeQuery(query);
```

Terza fase: ResultSet

- Un oggetto di tipo `ResultSet` è un oggetto che consente di analizzare (come una sorta di cursore o iteratore) la sequenza di tuple che costituiscono il risultato fornito da `executeQuery`
- Come cursore, un oggetto di tipo `ResultSet` è inizialmente posizionato **prima della prima riga (tupla)** del risultato
- L'oggetto di tipo `ResultSet` può poi avanzare col metodo `next()` che, una volta invocato, sposta il cursore di una tupla in avanti e restituisce `false` (booleano) se non ci sono più tuple da analizzare, `true` altrimenti

Terza fase: ResultSet (esempio)

```
String query="SELECT COF_NAME,PRICE FROM COFFEES";  
ResultSet rs=stmt.executeQuery(query);  
while (rs.next()) {  
    // elaborazione dei dati che si trovano  
    // nella tupla sotto esame  
}
```

→ Come si analizza la tupla “puntata” dall'oggetto rs?

Terza fase: i metodi getXXX

I metodi getXXX(y) (dove XXX denota un tipo) restituiscono il valore della colonna specificata dal parametro y corrispondente alla **riga corrente** (quella su cui è posizionato il cursore). Il parametro y può essere:

- il **nome** della colonna;
- il **numero d'ordine** della colonna

N.B. Il numero d'ordine è **quello della tabella dei risultati**.

XXX va sostituito con il tipo del dato che si vuole prelevare dal cursore nella posizione indicata dal parametro. Ovviamente tale tipo di dato deve essere compatibile con il tipo che il dato ha nella base di dati.

Terza fase: esempio di uso dei metodi

```
String query="SELECT COF_NAME,PRICE FROM COFFEES";  
ResultSet rs=stmt.executeQuery(query);  
while (rs.next()) {  
    String s=rs.getString("COF_NAME");  
    float n=rs.getFloat("PRICE");  
    System.out.println(s+" "+n);  
}
```

*in ORDINE DI
TUPLA IL VALORE
DELL'ATTRIBUTO*

Cosa stampa?

Terza fase: i metodi getXXX (cont.)

Istruzioni equivalenti alle precedenti righe all'interno del ciclo `while`:

```
String s=rs.getString(1);  
float n=rs.getFloat(2);
```

Abbiamo usato `getString()` e `getFloat()` per leggere dati dal `ResultSet()`. Esistono anche `getInt()`, `getDate()`, `getBoolean()`, `getTimestamp()` e altri.

Terza fase: corrispondenza tra tipi di dato Java e SQL

Tipo SQL	Tipo o Classe Java	Metodo di lettura di ResultSet
BIT	boolean	getBoolean()
CHAR (..)	String	getString()
VARCHAR (..)	String	getString()
DOUBLE	double	getDouble()
FLOAT	double	getDouble()
INTEGER	int	getInt()
NUMERIC	int	getInt()
REAL	float	getFloat()

Terza fase: corrispondenza tra tipi di dato Java e SQL

Tipo SQL	Tipo o Classe Java	Metodo di lettura di ResultSet
DATE	java.sql.Date	getDate()
TIME	java.sql.Time	getTime()
TIMESTAMP	java.sql.Timestamp	getTimestamp()

La corrispondenza fra i tipi SQL ed i tipi Java è comunque definita in maniera piuttosto “elastica”. Si noti, ad esempio, che per prelevare il prezzo di un caffè (tipo SQL `FLOAT`) dal result set dell’interrogazione effettuata, abbiamo in precedenza usato `getFloat()` al posto di `getDouble()`.

Il metodo `getDate()` restituisce un oggetto di classe `java.sql.Date` (che estende la classe `java.util.Date`) nel formato 'yyyy-MM-dd' (anno, mese, giorno). La classe `java.sql.Date` mette a disposizione alcuni metodi per eseguire elementari confronti sulle date (ad es., `compareTo`).

Molti altri metodi della classe sono però deprecati.

Nota sulla classe `java.sql.Date` (Cont.)

Per fare operazioni più complesse conviene trasformare l'oggetto di classe `java.sql.Date` in un oggetto di classe `java.util.GregorianCalendar`. Un possibile modo è riportato di seguito

```
String s = data.toString();  
int year=Integer.parseInt(s.substring(0,4));  
int month=Integer.parseInt(s.substring(5,7));  
int day=Integer.parseInt(s.substring(8,10));  
GregorianCalendar g= new GregorianCalendar(year,month,day);
```

Dove `data` è un oggetto di classe `java.sql.Date`.

Nota sulla classe `java.sql.Date` (Cont.)

Per ottenere una stampa da un `GregorianCalendar` in formato 'dd-mm-yyyy', si può ad esempio procedere come segue

```
System.out.print( g.get(Calendar.DAY_OF_MONTH));  
System.out.print( "-");  
System.out.print( g.get(Calendar.MONTH));  
System.out.print( "-");  
System.out.println(g.get(Calendar.YEAR));
```

Terza fase: metadati su un Result Set

È possibile ottenere i metadati (ad esempio, il numero di colonne, ecc.) su un oggetto di tipo `ResultSet` creando un oggetto di tipo `ResultSetMetaData` e invocando i metodi della classe `ResultSetMetaData` su di esso.

- `ResultSetMetaData getMetaData()`, metodo della classe `ResultSet`, restituisce i metadati dell'oggetto su cui è invocato
- `int getColumnCount()` restituisce il numero di colonne del `ResultSet` associato
- `String getColumnLabel(int column)` riceve un intero i come argomento, e restituisce il **nome** della i -esima colonna del `ResultSet` associato

Terza fase: esempio

Assumiamo di avere definito nella nostra base di dati la seguente tabella.

COF_NAME	SUP_ID	PRICE	SALES	TOTAL
Colombian	101	7.99	0	0
French Roast	49	8,99	0	0
Espresso	150	9.99	0	0
Colombian Decaf	101	8.99	0	0
French Roast Decaf	49	10.75	0	0

Terza fase: esempio di calcolo del result set

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(" SELECT * FROM COFFEES");
ResultSetMetaData rsmd = rs.getMetaData();
int numeroColonne = rsmd.getColumnCount();
for (int i = 1; i <= numeroColonne; i++) {
    if (i > 1) System.out.print(", ");
    String nomeColonna = rsmd.getColumnLabel(i);
    System.out.print(nomeColonna);
}
System.out.println("");
```

Cosa fa?

Terza fase: altri metodi di ResultSetMetaData

- `String getTableName (int column)` restituisce il nome della tabella a cui appartiene la colonna designata tramite il parametro
 - `String getSchemaName (int column)` restituisce il nome dello schema della tabella a cui appartiene la colonna designata tramite il parametro
- entrambi restituiscono la stringa vuota se il metodo non è applicabile

- `boolean isReadOnly (int column)` restituisce `true` se la colonna designata tramite il parametro **non è scrivibile**
- `boolean isWritable (int column)` restituisce `true` se la colonna designata tramite il parametro **è scrivibile**
- `boolean isSearchable (int column)` restituisce `true` se la colonna designata tramite il parametro **può essere usata in una clausola where**

Terza fase: Prepared Statement

Il “prepared statement” è una caratteristica dei DBMS basati su SQL. Con un prepared statement possiamo fornire ad un DBMS relazionale la definizione di un “comando parametrico”, ossia di una query o di un update in cui inseriamo dei punti interrogativi come parametri in posizioni in cui ci si aspetta dei dati. Tali parametri saranno poi istanziati nel momento in cui si invocherà lo statement per eseguirlo. Quando un DBMS elabora un prepared statement, non lo esegue, ma lo “compila” per sceglierne il piano di esecuzione, in modo che quando verrà invocato disporrà dell’algoritmo per eseguirlo.

In JDBC:

- Un PreparedStatement è una specializzazione di Statement
- Contiene una query, dalla struttura fissata, ma in grado di ricevere parametri che sono istanziati a run-time
- Può quindi essere usata più volte con effetti diversi

Terza fase: esempio

```
PreparedStatement updateSales;  
String updateString="UPDATE COFFEES SET SALES=? " +  
    "WHERE COF_NAME=?";  
updateSales=conn.prepareStatement(updateString);  
int[] salesForWeek={175,150,60,115,90};  
String[] coffees={"Colombian","French Roast","Espresso",  
    "Colombian Decaf","French Roast Decaf"};  
int len=coffees.length;  
for (int i = 0; i < len; i++) {  
    updateSales.setInt(1,salesForWeek[i]);  
    updateSales.setString(2,coffees[i]);  
    updateSales.executeUpdate();  
}
```

Terza fase: esempio

COF_NAME	SUP_ID	PRICE	SALES	TOTAL
Colombian	101	7.99	175	0
French Roast	49	8.99	150	0
Espresso	150	9.99	60	0
Colombian Decaf	101	8.99	115	0
French Roast Decaf	49	10.75	90	0

}

Terza fase: note sui metodi setXXX

- Per sapere quali metodi `setXXX` usare in modo da impostare correttamente i parametri di un prepared statement, fate riferimento alla tabella di corrispondenza riportata in precedenza per i metodi `getXXX`. Valgono le stesse regole di “elasticità”
- In particolare, quando dovete inserire un valore decimale che in SQL corrisponda ad un Float, Double o Real, si consiglia l’uso di `setDouble()`
- Il metodo `setDate(Date d)` vuole in input un oggetto della classe `java.sql.Date`. Per creare un oggetto di questo tipo potete utilizzare il metodo statico `valueOf (String s)` della classe `java.sql.Date` in cui la stringa passata come parametro abbia il formato ‘yyyy-MM-dd’ (anno-mese-giorno). Ad esempio,
`java.sql.Date data=java.sql.Date.valueOf("2000-01-12");`

Terza fase: differenza tra statement e prepared statement

Illustreremo ora un esempio che sottolinea la differenza tra statement e prepared statement nell'utilizzo di JDBC.

Prima di farlo, ricordiamo che

- quando usiamo uno statement, noi confezioniamo una stringa che rappresenta una istruzione SQL e poi mandiamo al DBMS tale stringa allo scopo di eseguire l'istruzione.
- quando usiamo un prepared statement, la stringa che confezioniamo **non** è una istruzione completa, ma una istruzione parametrica, in cui ci sono dei simboli speciali (il ?) al posto di porzioni della istruzione che rappresentano dei valori mancanti. Quando mandiamo al DBMS la stringa, il sistema **non** esegue l'istruzione (non potrebbe, visto che non è completa), ma la "compila" al fine di derivare il cosiddetto "piano di esecuzione". L'esecuzione dell'istruzione avviene al momento in cui si chiede al DBMS di eseguire lo statement utilizzando il meccanismo che sostituisce il simbolo speciale con i valori concreti che avremo preparato.

Terza fase: differenza tra statement e prepared statement

Riconsideriamo l'esempio visto prima trasformando le istruzioni:

```
ResultSet rs;
```

```
rs = st.executeQuery("SELECT nomepers FROM persona " +
                    "WHERE citta='napoli'");
```

in

acquisisci da un pagina web la stringa s inserita dall'utente

```
ResultSet rs;
```

```
rs = st.executeQuery("SELECT nomepers FROM persona " +
                    "WHERE citta=' " + s + " '");
```

Ovviamente, se l'utente ci fornisce `napoli` come stringa `s`, otteniamo esattamente lo stesso comportamento di prima, poichè il comando mandato a SQL è: `SELECT nomepers FROM persona WHERE citta='napoli'`.

Quindi, in queste condizioni, il programma stamperà in output ancora:

ugo

anna

Terza fase: differenza tra statement e prepared statement

Ma supponiamo che l'utente fornisca come stringa s:

```
' OR 1=1 --
```

Adesso il comando che il programma manda ad SQL diventa:

```
SELECT nomepers FROM persona WHERE citta=' ' OR 1=1 -- '
```

Poichè il simbolo `--` è usato in SQL per iniziare un commento, il compilatore SQL lo ignora e ignora tutto ciò che segue tale simbolo. Ne consegue che in questo caso il programma stamperà in output:

ugo
anna
aldo

Abbiamo così ottenuto dalla base di dati una informazione (il nome di tutte le persone) che magari non era lecito rivelare. Quello appena mostrato è un esempio di **SQL injection**, un tipo di attacco software molto comune che ha lo scopo di “rubare” dati privati, facendo leva sul meccanismo degli “statement” SQL invocati da software (in questo caso da JDBC).

Terza fase: differenza tra statement e prepared statement

Consideriamo ancora il caso che l'utente fornisca come stringa s:

```
' OR 1=1 --
```

ma adesso assumiamo di avere scritto il programma in questo modo:

[illegible]

```
pst.setString(1,s);
pst.executeQuery();
```

L'ultimo comando ha l'effetto di eseguire la query

```
SELECT nomepers FROM persona WHERE citta= α
```

con α che viene istanziato con la stringa s. Questa query, ovviamente, non restituisce alcun risultato.

Abbiamo così sventato il pericolo di eseguire una query che poteva essere pericolosa rispetto alla sicurezza dei dati nella base di dati.

Terza fase: CallableStatement

- L'interfaccia CallableStatement, sottoclasse di PreparedStatement, permette di chiamare stored procedure
- Come per gli oggetti della classe PreparedStatement, è possibile usare il punto interrogativo per indicare un parametro che sarà istanziato a run-time

Terza fase: CallableStatement (esempio)

Supponiamo di aver definito la seguente

```
CREATE FUNCTION PersoneNateInCitta(c VARCHAR(25))
```

...

```
SELECT nomepers FROM persona
```

WHERE città=c;

$$\left(\begin{array}{c} \vdots \end{array} \right)$$

RETURN QUERY ...

Nel programma Java possiamo scrivere:

```
CallableStatement cstmt = conn.prepareCall(" call PersoneNatInCitta(?)");
```

```
ResultSet rs;
```

```
for (int i = 0; i < 3; i++) {
```

```
cstmt.setString(1,citta[i]);
```

```
rs = cstmt.executeQuery();
```

```
while (rs.next()) ....
```

}

Quarta fase: chiusura della connessione

Dopo le fasi:

- **Prima fase:** caricamento dinamico della classe corrispondente al driver JDBC
- **Seconda fase:** connessione con la sorgente di dati
- **Terza fase:** sessione di interazione attraverso la connessione, in cui si definiscono gli “statement” e si eseguono tali statement comunicandoli alla sorgente

nel momento in cui si decide che l'esigenza di accesso alla base di dati è esaurita, si procede alla **quarta fase**, quella in cui si chiude la connessione. Gli oggetti di tipo `Connection`, `Statement`, `ResultSet`, che evidentemente non si vogliono più utilizzare, devono essere **chiusi** con il metodo `close()`.